Algorithmique avancée

Langage C

Laurent Bellon - 2026-2027

Cours Avancé en Algorithmique

Rappels et Introduction

Théorie — Complexité Algorithmique

Analyse de cas (pire, meilleur, moyen)

L’analyse de la complexité d’un algorithme ne se limite pas à une seule mesure. Elle considère principalement trois scénarios qui reflètent des comportements différents selon la nature des données d’entrée :

- **Meilleur cas**
- **Pire cas**
- **Cas moyen**

La compréhension de ces cas permet d’évaluer de manière plus réaliste l’efficacité d’un algorithme dans diverses situations.

Définitions : Pire Cas et Meilleur Cas

Le Pire Cas (Worst Case)

- Représente la situation où l'algorithme effectue **le nombre maximal d'opérations** possibles pour une entrée de taille (n).
- **Utile pour :** Garantir un temps d'exécution maximal, crucial dans les applications critiques.
- **Notation :** Souvent noté avec la borne supérieure (O(f(n))).

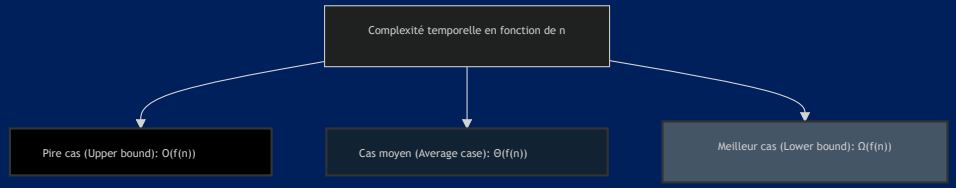
Le Meilleur Cas (Best Case)

- Correspond à la situation où l'algorithme effectue **le nombre minimal d'opérations**.
- **Usage :** Rarement utilisé seul car peu représentatif du comportement moyen, mais permet parfois des optimisations.
- **Notation :** Souvent noté avec la borne inférieure (Omega(f(n))).

Définitions : Cas Moyen et Synthèse Visuelle

Le Cas Moyen (Average Case)

- Exprime la **complexité moyenne** sur toutes les entrées possibles de taille (n).
- **Calcul :** Plus difficile car nécessite de connaître la distribution des entrées.
- **Notation :** S'approxime souvent par (Theta(f(n))) si la distribution est connue.



Exemples Illustratifs : Recherche Linéaire

Considérons une liste non triée de taille (n) et une fonction cherchant un élément (x).

```
int recherche_lineaire(int tab[], int n, int x) {
    for (int i = 0; i < n; i++) {
        if (tab[i] == x) return i; // retour dès la première occurrence
    }
    return -1; // si pas trouvé
}
```

Résumé

Cas	Complexité	Explication
Meilleur cas	$\Omega(1)$	x est en première position
Pire cas	$O(n)$	x est absent ou en dernière position
Cas moyen	$\Theta(n)$	En moyenne, x est trouvé à la moitié de la liste

Exemples Illustratifs : Tri à Bulles (Bubble Sort)

```c
// Fonction pour échanger deux éléments
void echanger(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Fonction de tri à bulle
void bubbleSort(int tableau[], int taille) {
    int i, j;
    for (i = 0; i < taille - 1; i++) {
        for (j = 0; j < taille - i - 1; j++) {
            if (tableau[j] > tableau[j + 1]) {
                echanger(&tableau[j], &tableau[j + 1]);
            }
        }
    }
}
```

Résumé

Cas	Complexité	Explication
Meilleur cas	$\Omega(n)$	Le tableau est déjà trié.
Pire cas	$O(n^2)$	Le tableau est trié à l'envers
Cas moyen	$\Theta(n^2)$	Par analyse statistique des permutations aléatoires

Points Importants & Références

Ce qu’il faut retenir

- **Le pire cas** est omniprésent dans la littérature car il garantit une capacité maximale à gérer toutes les entrées.
- **Le meilleur cas** est souvent négligé car trop optimiste (ex: liste déjà triée pour certains tris).
- **Le cas moyen** est le plus représentatif mais parfois complexe à calculer, nécessitant des hypothèses sur la distribution des données.
- Les notations Big O, Big Ω et Big Θ sont respectivement liées au pire cas, au meilleur cas, et au cas moyen.

Sources Consultées

- Univ. de la Réunion - Complexité des algorithmes
- Cours PDF - Université d’Ottawa
- MarcArea - Complexité algorithmique et notation Grand O
- JP Zanotti - Algorithmique et complexité
- LRI - Algorithmique et complexité (PDF)

Cours Avancé en Algorithmique

Rappels et Introduction : Complexité Algorithmique

L’Analyse de Complexité, C’est Quoi ?

- **Évaluer l’efficacité** d’un algorithme en comptant le nombre d’opérations.
- La mesure dépend de la taille de l’entrée (n).
- L’approche d’analyse diffère pour les algorithmes **itératifs** (boucles) et **récursifs** (appels à soi-même).

Complexité des Algorithmes Itératifs

Principes Fondamentaux

- S’appuie sur des boucles simples ou imbriquées.
- Le calcul consiste à **compter le nombre d’itérations**.
- Exemple : une boucle simple exécute (O(n)) opérations.

Exemples Concrets

1. Somme Itérative (Boucle simple)

```
void somme_iterative(int n) {
    for (int i = 1; i ⩽ n; i++) {
        // opération constante
    }
}
```

- Complexité : (T(n) = O(n))

Exemples Concrets

2. Boucles Imbriquées (Produit Itératif)

```
void produit_iteratif(int n) {
    for (int i = 1; i ⩽ n; i++) {
        for (int j = 1; j ⩽ n; j++) {
            // opération constante
        }
    }
}
```

- Complexité : (T(n) = O(n^2))

Complexité des Algorithmes Récursifs

Le Principe des Relations de Récurrence

- La complexité $T(n)$ est définie en fonction de T de sous-problèmes plus petits.
- L'analyse passe par la **résolution de ces relations**.

Exemple simple : Somme récursive

```c
int somme_recursive(int n) {
    if (n == 0) return 0;
    return n + somme_recursive(n - 1);
}
```

- **Relation de récurrence :** (T(n) = T(n-1) + O(1)), avec (T(0) = O(1))
- **Résolution :** (T(n) = O(n))

Cas similaire : Factorielle

- La complexité de la factorielle récursive est également en (O(n)).

Complexité Récursive

Cas du Tri Fusion (Algorithme "Diviser pour Régner")

```
void tri_fusion(int tab[], int deb, int fin) {
    // ...
    tri_fusion(tab, deb, milieu);
    tri_fusion(tab, milieu + 1, fin);
    fusion(tab, deb, milieu, fin); // fusion en O(n)
}
```

- **Relation de récurrence :** $T(n) = 2 * T(n/2) + O(n)$
- **Résolution (Théorème de Maître) :** ($T(n) = O(n \log n)$)

Théorême de maître

Cas	Condition sur **f(n)**	Qui domine ?	Résultat
1	$f(n) = O(n^{log_b a - \epsilon})$ (f(n) plus petit)	La **récursion** (les sous-problèmes)	$T(n) = O(n^{log_b a})$
2	$f(n) = O(n^{log_b a})$ (f(n) du même ordre)	Équilibre entre récursion et fusion	$T(n) = O(n^{\log_b a} \cdot \log n)$
3	$f(n) = O(n^{\log_b a + \epsilon})$ (f(n) plus grand)	Le **travail hors récursion** (f(n))	$T(n) = O(f(n))$

Théorême de maître

En résumé : le théorème de maître, c'est un tableau de recettes qui dit :

- « Comment on divise le problème »
- « Combien ça coûte en plus ? »

Pour l'exemple :

- a = 2 (on crée 2 sous-problèmes)
- b = 2 (chaque sous-problème fait la moitié de la taille)
- f(n) = O(n)

donc :

- La récursion toute seule coûterait environ **n log n**
- Le travail hors récursion (la fusion) coûte n

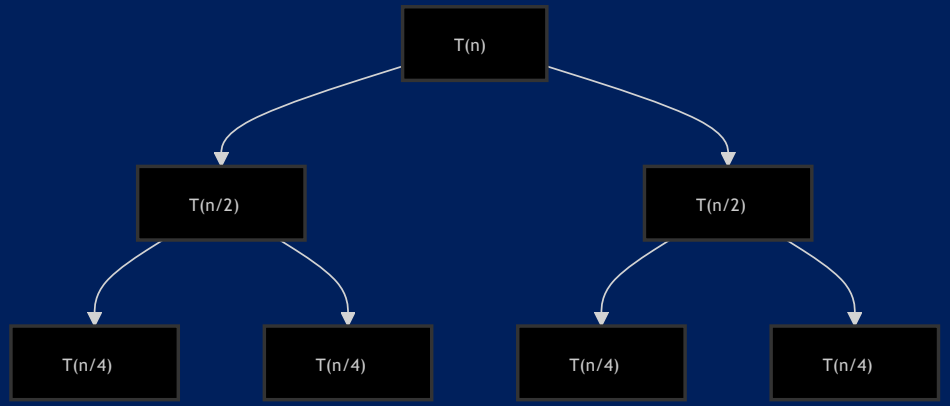
On est donc dans le cas 2 → résultat :

T(n)=O(n log n)

Techniques de Résolution des Relations de Récurrence

1. **Méthode d’expansion :** déplier la récursion.
2. **Théorème de maître (Master theorem) :** formules pour certains cas.
3. **Arbre de récursion :** visualiser les appels et les coûts.

Arbre de Récursion du Tri Fusion



Récursion vs. Itération

Aspect	Itération	Récursion
Écriture	Boucles (for, while)	Appels de fonction récursive
Complexité temporelle	Généralement plus directe	Analyse par équations de récurrence
Complexité en mémoire	Constante ou faible	Peut être plus élevée (pile d’appels)
Utilisation pratique	Simple ou complexe	Naturel pour problèmes divisés

En Résumé

- L’**itération** est souvent plus simple à analyser et plus efficiente en mémoire.
- La **récursion** offre des solutions élégantes pour certains problèmes (ex: diviser pour régner) mais nécessite une analyse de complexité plus élaborée, incluant la gestion de la pile d’appels.

Ce qu’il faut retenir & Ressources

Ce qu’il faut retenir

- L’analyse de complexité est cruciale pour évaluer les performances.
- Les algorithmes **itératifs** se caractérisent par le comptage des boucles.
- Les algorithmes **récursifs** demandent la mise en place et la résolution de **relations de récurrence** via des méthodes spécifiques (expansion, Théorème de Maître, arbre de récursion).
- La maîtrise de ces techniques est un atout majeur en algorithmique avancée.

Sources consultées

- Geeks for Geeks - Time Complexity analysis of recursive/iterative algorithms
- MIT OpenCourseWare - Introduction to Algorithms
- Stanford CS Education Library - Recurrence Relations and Master Theorem
- Khan Academy - Recursive Algorithms
- Wikipedia - Recurrence relation

Choix de structures de données : Tableau vs Liste Chaînée

Comprendre leurs compromis pour des implémentations efficaces

Le choix d’une structure de données impacte directement la performance, la gestion mémoire et la simplicité d’implémentation. Deux structures fondamentales sont le **tableau (array)** et la **liste chaînée (linked list)**.

Définitions

- **Tableau (Array) :**
  - Contigu en mémoire.
  - Accès direct en temps constant (O(1)) via index.
  - Taille généralement fixée à la création.
- **Liste chaînée (Linked List) :**
  - Chaîne d’éléments (noeuds) reliés par des pointeurs/références.
  - Taille dynamique.
  - Insertion/suppression aisée.

Tableau vs Liste Chaînée : Comparaison Détaillée

Critère	Tableau	Liste chaînée
Accès aléatoire	(O(1)) — accès direct par index	(O(n)) — doit parcourir la liste
Insertion	(O(n)) — décalage possible	(O(1)) — simple ré-affectation de pointeurs
Suppression	(O(n)) — décalage possible	(O(1)) — déconnexion du noeud
Gestion mémoire	Mémoire contiguë, overhead faible	Overhead mémoire important (pointeurs)
Taille	Fixe ou redimensionnement coûteux	Dynamique, flexible
Localité mémoire	Excellente, favorise cache CPU	Fragile — discontinuité mémoire
Complexité de la structure	Simple	Complexe (pointeurs, gestion des liens)

Exemples Concrets : Performances des Opérations

Accès direct dans un tableau

```c
int tab[5] = {1,2,3,4,5};
int val = tab[3]; // Accès direct en O(1)
```

- Accès ultra-rapide grâce à la contiguïté mémoire.

Insertion en début d’une liste chaînée

```c
typedef struct Node {
    int data;
    struct Node* next;
} Node;

void inserer_debut(Node** head, int val) {
    Node* nouveau = malloc(sizeof(Node));
    nouveau->data = val;
    nouveau->next = *head;
    *head = nouveau;
}
```

- Insérer un élément en début se fait en temps (O(1)), sans déplacement d’éléments.

Illustration Mémoire : Contiguïté vs Liens

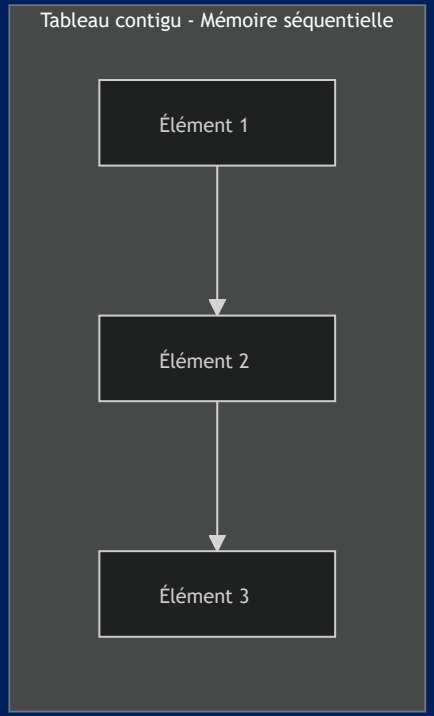
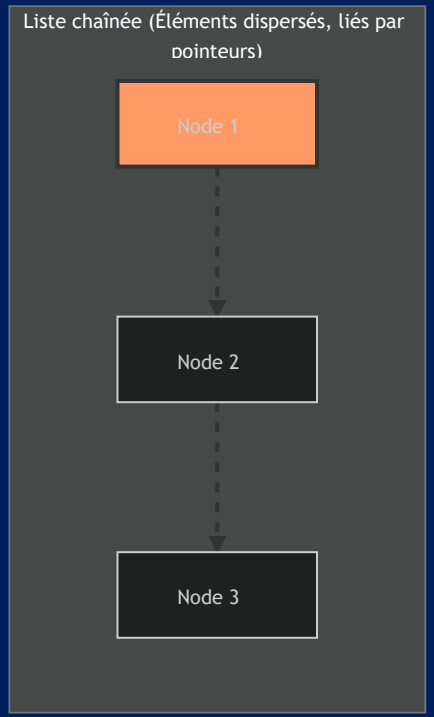


Illustration Mémoire : Contiguïté vs Liens



- **Tableau :** Les éléments sont adjacents, optimisant le cache CPU.
- **Liste chaînée :** Les éléments peuvent être dispersés en mémoire, nécessitant des pointeurs.

Choix Stratégique selon les Cas d’Usage

- **Quand privilégier le Tableau :**
  - La taille est connue ou ne varie que peu.
  - Accès fréquent par index (recherche rapide).
  - Exécution nécessitant rapidité mémoire et CPU (ex : calcul matriciel, vecteurs).
- **Quand privilégier la Liste Chaînée :**
  - La taille varie fréquemment (ajouts/suppressions régulières).
  - Les insertions/suppressions sont fréquentes, mais l’accès par index est rare.
  - Exemples : files d’attente (queues), piles (stacks), gestion dynamique de données.

Conclusion et Ressources

Ce qu’il faut retenir

Privilégier le tableau ou la liste chaînée dépend des besoins en accès rapide, gestion dynamique de la mémoire et fréquence des opérations d’insertion/suppression. Une bonne compréhension des compromis inspire des choix optimisés en algorithmique avancée.

Sources et lectures complémentaires

- GeeksforGeeks – Array vs Linked List
- Medium – When to use Linked List over Array
- Wikipedia – Data Structure
- Guru99 – Arrays and Linked Lists
- Computer Science Field Guide – Arrays and Linked Lists

Rappels : Pile et Tas (Stack et Heap)

Gestion Mémoire Avancée

- Dans la gestion mémoire d’un programme (langage C et la plupart des langages compilés), deux régions clés stockent les données :
  - **La pile (stack)**
  - **Le tas (heap)**
- Comprendre leurs caractéristiques permet d’optimiser la gestion mémoire, la sécurité et la performance.

La Pile (Stack) : Mémoire Automatique

Description & Comportement

- Zone mémoire **statique** et **structurée** en mode LIFO (Last In, First Out).
- Gère les variables locales, les paramètres de fonctions et le contexte d’appel.
- Allocation et désallocation automatiques à l’entrée et sortie de blocs/fonctions.

Caractéristiques clés

Caractéristique	Description
Allocation	Automatique
Taille	Limitée (~1Mo à quelques Mo)
Accès	Très rapide, adressage contigu
Durée de vie	Cycle d’une fonction/portée locale
Sécurité	Risque de débordement (stack overflow)

Le Tas (Heap) : Mémoire Dynamique

Description & Comportement

- Zone mémoire **dynamique** gérée manuellement via des appels d'allocation (`malloc` , `calloc`) et de libération (`free`).
- Permet d'allouer des espaces mémoire dont la taille peut être décidée à l'exécution.

Caractéristiques clés

Caractéristique	Description
Allocation	Manuelle (via malloc/free)
Taille	Plus grande, limitée surtout par mémoire physique disponible
Accès	Plus lent que la pile (fragmentation, non contiguë)
Durée de vie	Contrôlée par le programme
Sécurité	Risque de fuites mémoire, déréférencement

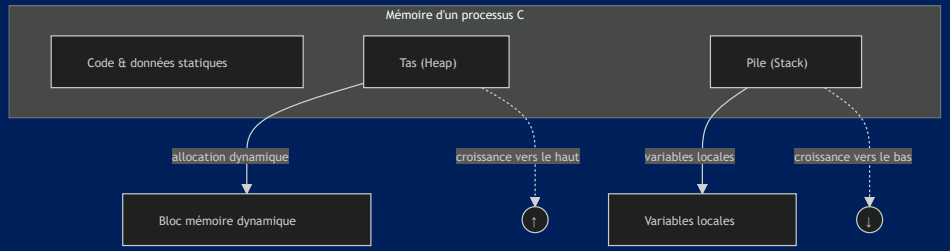
Pile vs Tas

Choix Stratégique selon les Besoins

Critère	Pile (Stack)	Tas (Heap)
Allocation	Automatique	Manuelle (malloc/free)
Vitesse d’allocation	Très rapide	Plus lente
Taille	Limitée	Plus importante
Fragmentation	Pas de fragmentation	Possible fragmentation mémoire
Durée de vie des données	Limité à la portée locale	Contrôlable globalement
Erreurs fréquentes	Débordement pile	Fuites mémoire, double free

Visualisation de la Gestion Mémoire

Comment les deux régions interagissent



Synthèse & Bonnes Pratiques

Optimisation et Stabilité des Programmes

- Pour des allocations temporaires et taille connue, privilégier la **pile** (variables locales).
- Pour des données dont la taille est dynamique ou persistante, utiliser le **tas**.
- Toujours libérer la mémoire allouée sur le tas pour éviter les fuites (ex.: utiliser `free`).
- Attention aux dépassements de pile (variables trop volumineuses) et aux erreurs sur le tas (déréférencement de pointeurs invalides, fuites).

Sources consultées

- GeeksforGeeks – Stack vs Heap
- Medium – Understanding Stack and Heap Memory in C
- Wikipedia – Stack (computer science)
- Wikipedia – Heap (data structure)
- TutorialsPoint – Stack vs Heap

Optimisation et Gestion Mémoire Avancée

Les pointeurs en langage C sont essentiels pour la gestion mémoire et la flexibilité du code.

L’usage avancé, notamment des **pointeurs de fonctions**, permet des programmes modulaires, optimisés et dynamiques.

Pointeurs Avancés : Au-delà de l’Adresse simple

Notion de pointeur de pointeur

Un pointeur peut lui-même désigner un autre pointeur. Utile pour manipuler des tableaux dynamiques à plusieurs dimensions ou modifier des adresses dans des fonctions.

```
int a = 10;
int *ptr = &a;      // pointeur sur int
int **pptr = &ptr;  // pointeur sur pointeur sur int
```

Accès via : `**pptr` *pour obtenir la valeur 10.*

Pointeurs et tableaux dynamiques

Pour une matrice, l'allocation et l'accès se font via un pointeur de pointeur.

```
int **matrice = malloc(rows * sizeof(int *));
for (size_t i = 0; i < rows; i++) {
    matrice[i] = malloc(cols * sizeof(int));
}
```

La libération mémoire correspondante est requise.

Pointeurs de Fonctions : Flexibilité & Modularité

Déclaration et utilisation

Un pointeur de fonction stocke l'adresse d'une fonction, permettant de l'appeler via ce pointeur.

```
int addition(int a, int b) { return a + b; }

int (*ptr_fonction)(int, int) = &addition;

int resultat = ptr_fonction(5, 3); // resultat = 8
```

Usage en programmation

- **Callbacks :** Fonctions passées en argument pour personnaliser des algorithmes.
- **Tables de dispatch :** Sélection dynamique de fonctions à exécuter.
- **Gestion d'état et événements :** Dans des systèmes embarqués.

Exemple Pratique : Tri avec pointeurs de fonctions

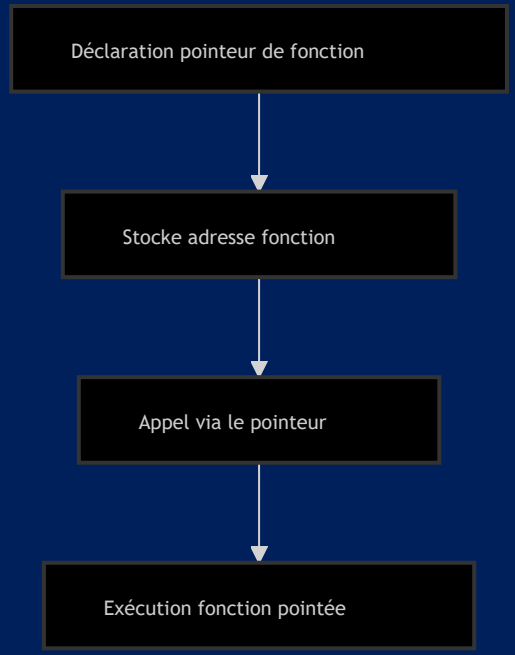
Le pointeur de fonction permet de rendre un algorithme de tri générique en changeant la logique de comparaison.

```c
// Fonctions de comparaison
int comparer_asc(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}
int comparer_desc(const void *a, const void *b) {
    return (*(int*)b - *(int*)a);
}

// Algorithme de tri générique
void tri_generique(int *tab, size_t n, int (*cmp)(const void*, const void*)) {
    for (size_t i = 0; i < n - 1; i++) {
        for (size_t j = i + 1; j < n; j++) {
            if (cmp(&tab[i], &tab[j]) > 0) { // Utilisation du pointeur de fonction
                int temp = tab[i];
                tab[i] = tab[j];
                tab[j] = temp;
            }
        }
    }
}
```

Appel : `tri_generique(tab, n, comparer_asc);`

Comprendre les pointeurs de fonctions



Risques et bonnes pratiques

- **Nullité du pointeur :** Toujours vérifier un pointeur de fonction avant usage.
- **Correspondance signature :** Le prototype doit correspondre strictement.
- **Gestion de la mémoire :** S'assurer de la validité des pointeurs manipulés.
- **Lisibilité :** Commenter les pointeurs complexes pour éviter la confusion.

En Résumé

Ce qu’il faut retenir

La maîtrise des pointeurs, notamment des pointeurs de fonctions, apporte une **flexibilité essentielle** à l’écriture de logiciels **modulaires et performants**. Elle permet des techniques avancées d’abstraction et d’optimisation en C.

Sources et approfondissements

- TutorialsPoint — Pointers in C
- GeeksforGeeks — Function Pointers in C
- CProgramming — Using Function Pointers
- Wikipedia — Pointer (computer programming)
- Modern C — Advanced Pointers

Introduction à la Gestion Mémoire Dynamique en C

La gestion dynamique de la mémoire en C est un pilier essentiel pour des programmes performants et stables. Elle repose sur quatre fonctions clés :

- `malloc` : Allouer de la mémoire non initialisée.
- `calloc` : Allouer de la mémoire et l'initialiser à zéro.
- `realloc` : Redimensionner un bloc de mémoire existant.
- `free` : Libérer la mémoire allouée dynamiquement.

Enjeu : Maîtriser ces fonctions est indispensable pour éviter les fuites mémoire et les comportements indéfinis.

Les Fondamentaux : `malloc`, `calloc`, `realloc`, `free`

Fonction	Usage	Signature	Particularités
`malloc`	Allouer un bloc mémoire brut, non initialisé	`void *malloc(size_t size);`	Retourne un pointeur vers la mémoire non initialisée ; `NULL` si échec
`calloc`	Allouer et initialiser à 0	`void *calloc(size_t nmemb, size_t size);`	Initialise chaque octet à 0
`realloc`	Redimensionner un bloc mémoire	`void *realloc(void *ptr, size_t size);`	Peut déplacer le bloc en mémoire ; copie éventuellement les données
`free`	Libérer un bloc mémoire	`void free(void *ptr);`	Accepte `NULL` sans effet, doit être appelé pour chaque allocation

Bonnes Pratiques d’Utilisation

1. Initialisation des pointeurs : Toujours initialiser les pointeurs dynamiques à `NULL` avant allocation.

```
int *ptr = NULL;
ptr = malloc(sizeof(int) * 10);
```

2. Vérification d’allocation : Toujours tester le retour de `malloc`, `calloc` ou `realloc`.

```
if (ptr == NULL) { /* gestion erreur */ }
```

Pour `realloc`, utiliser une variable temporaire :

```
void *temp = realloc(ptr, new_size);
if (temp ≠ NULL) { ptr = temp; } else { /* gérer erreur, ptr est toujours valide */ }
```

Bonnes Pratiques d'Utilisation

3. Libération systématique : Ne jamais oublier d'appeler `free` pour chaque bloc alloué.

```
free(ptr); // Évite les fuites mémoire
```

4. Neutraliser les pointeurs libérés : Après `free`, mettre le pointeur à `NULL` pour éviter les pointeurs sauvages.

```
ptr = NULL; // Empêche l'usage erroné post-libération
```

Erreurs Courantes à Éviter

1. Fuites mémoire : Oublier de libérer un bloc alloué ou perdre le pointeur (ex: écrasement).

```
ptr = malloc(100);
// ...
ptr = malloc(200); // Ancienne allocation perdue si non libérée
```

2. Pointeurs sauvages (Dangling Pointers) : Utiliser un pointeur après que la mémoire qu'il désigne a été libérée.

```
free(ptr);
*ptr = 42; // Comportement indéfini !
```

Erreurs Courantes à Éviter

3. Double libération (Double Free) : Libérer deux fois le même bloc de mémoire. Peut provoquer une erreur de segmentation.

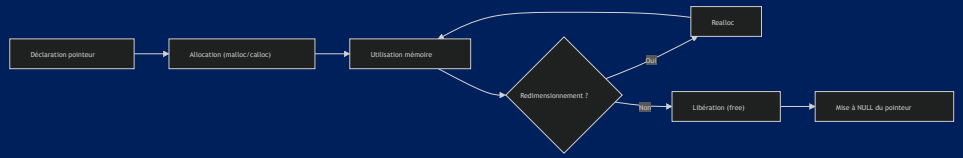
```
free(ptr);
// ...
free(ptr); // Erreur !
```

4. Mauvais usage de `realloc` : Réaffecter directement le résultat de `realloc` sans sauvegarder. En cas d'échec, le pointeur original est perdu.

```
ptr = realloc(ptr, new_size); // Risqué en cas d'échec
// Utilisez la bonne pratique de la slide précédente !
```

Cycle de Vie de la Mémoire Dynamique & Application Pratique

Comprendre le flux de gestion mémoire est crucial pour des applications robustes.



Exemple complet :

La mise en œuvre de ces pratiques est démontrée dans un exemple de code complet qui alloue, utilise, redimensionne et libère un tableau d'entiers, en incluant la gestion des erreurs. C'est la synthèse de toutes les bonnes pratiques présentées.

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *tab = NULL;
    size_t taille = 5;

    // Allocation initiale
    tab = malloc(taille * sizeof(int));
    if (!tab) {
        perror("malloc failed");
        return EXIT_FAILURE;
    }
```

–> Suite -->

```c
    // Utilisation
    for (size_t i = 0; i < taille; i++) tab[i] = i;

    // Redimensionnement
    int *temp = realloc(tab, 2 * taille * sizeof(int));
    if (!temp) {
        free(tab);
        perror("realloc failed");
        return EXIT_FAILURE;
    }
    tab = temp;

    // Initialisation nouvelles cases
    for (size_t i = taille; i < 2 * taille; i++) tab[i] = 0;

    // Libération
    free(tab);
    tab = NULL;

    return EXIT_SUCCESS;
}
```

Ce qu’il Faut Retenir & Ressources

Conclusion : La rigueur dans l’utilisation des fonctions d’allocation dynamique et la bonne gestion des pointeurs associée sont les clés pour éviter fuites mémoire, erreurs et garantir la stabilité des programmes en C.

Sources consultées :

- GNU C Library Documentation – malloc, calloc, realloc, free
- GeeksforGeeks - Dynamic Memory Allocation in C
- Stack Overflow – Common Mistakes with malloc and free
- Wikipedia – Dynamic Memory Allocation
- Linux Journal – Memory Management: malloc, calloc, realloc and free

Rappels et Introduction : Optimisation et Gestion Mémoire Avancée

Les standards C17 et C23 révolutionnent la gestion mémoire et la sûreté du code. Ces évolutions sont cruciales pour améliorer la robustesse, la portabilité et les performances des applications.

C17 (2018) : Stabilisation et Précision

C17 a principalement agi comme une version corrective pour C11, visant à renforcer la fiabilité du langage.

Points clés :

- **Correction de bugs et ambiguïtés** : Notamment en multithreading et fonctions standards.
- **Stabilisation des fonctions standards** : `memcpy`, `memset` sont mieux définies pour éviter les comportements indéfinis en bordure mémoire.
- **Renforcement des règles** : Meilleure définition des comportements indéfinis, en particulier autour des pointeurs.

Impact majeur : Amélioration significative de la sûreté d'exécution grâce à des définitions plus claires qui réduisent les erreurs mémoire.

C23 : Vers une sûreté accrue et nouveautés

C23, actuellement en définition, promet des avancées majeures pour la sûreté et la robustesse du code.

Fonctionnalité	Impact sur gestion mémoire et sûreté
Initialisation stricte des variables	Réduction des risques liés aux variables non initialisées.
Allocation automatique avec `malloc_array`	Nouvelle fonction sûre d'allocation de tableaux, prévient les débordements de taille.
Déclarations imbriquées dans les blocs	Facilite l'écriture de code plus sûr, évite la capture involontaire de variables.
Améliorations des macros	Permettent d'éviter les effets de bord non contrôlés, souvent source de bugs.
Support renforcé `_Checked`	Amélioration des garanties sur les pointeurs et les accès mémoire (projet Csafe).

`malloc_array` : L'Allocation Sûre en Pratique

L'un des apports les plus significatifs de C23 est `malloc_array`, qui adresse une vulnérabilité courante.

Avant C23 : Risque de dépassement silencieux

```
int *tab = malloc(n * sizeof(int));
// Si n * sizeof(int) > SIZE_MAX, un débordement silencieux peut survenir.
```

Avec C23 : Allocation protégée

```
int *tab = malloc_array(n, sizeof(int));
if (!tab) {
    // Gestion d'erreur plus sûre, évite débordement de taille.
}
```

Cette fonction empêche les débordements de taille lors du calcul de l'espace total nécessaire, renforçant la sécurité de l'allocation dynamique.

Améliorations Complémentaires et Cycle d’Évolution

C23 intègre d’autres avancées pour la robustesse et la compatibilité.

- **Macros conditionnelles** : `__has_include` et `__has_builtin` pour une meilleure portabilité et optimisation.
- **Bibliothèque standard** : Modifications pour garantir un fonctionnement sécurisé et conforme.
- **Compilateurs** : Amélioration des warnings pour une meilleure détection des erreurs mémoire dès la compilation.

Bilan et Références

Ce qu’il faut retenir :

Les évolutions du langage C via C17 et C23 visent principalement à offrir une base plus robuste et sécurisée, limitant les erreurs mémoire courantes (dépassements, pointeurs non initialisés) et ajoutant des fonctionnalités pour une allocation plus sûre, contribuant à la fiabilité et à la maintenabilité du code système et bas niveau.

Sources consultées :

- ISO/IEC 9899:2018 - Draft for C17
- WG14 Working Group on C Standard - Drafts and proposals
- Microsoft C Blog – C23 and its new features
- CppReference – C23 New Library Functions
- Red Hat Developer – C23 malloc_array and other features

Section 2 : Structures de données dynamiques avancées

Listes Doublement Chaînées et Circulaires : Principes et Nœuds

Les listes doublement chaînées améliorent les listes simplement chaînées par une **navigation bidirectionnelle** aisée entre les nœuds.

La version circulaire intègre une boucle où le **dernier nœud pointe vers le premier** et inversement.

Structure des Nœuds

Chaque nœud d'une liste doublement chaînée contient :

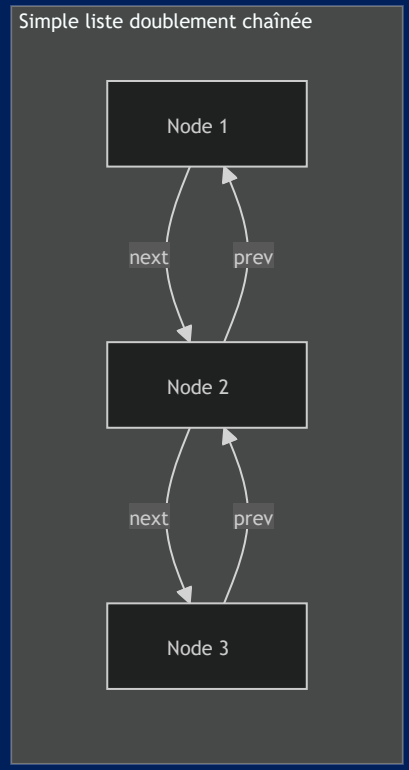
- Un champ **donnée** (payload)
- Un pointeur vers le nœud **précédent** (`prev`)
- Un pointeur vers le nœud **suivant** (`next`)

Définition typique en C :

```c
typedef struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
} Node;
```

Fonctionnement :

- Le pointeur `prev` permet la navigation inverse, facilitant insertion/suppression sans parcours préalable.
- Le pointeur `next` maintient le lien avant.
- En liste non circulaire : `prev` du premier nœud est `NULL` , `next` du dernier nœud est `NULL` .

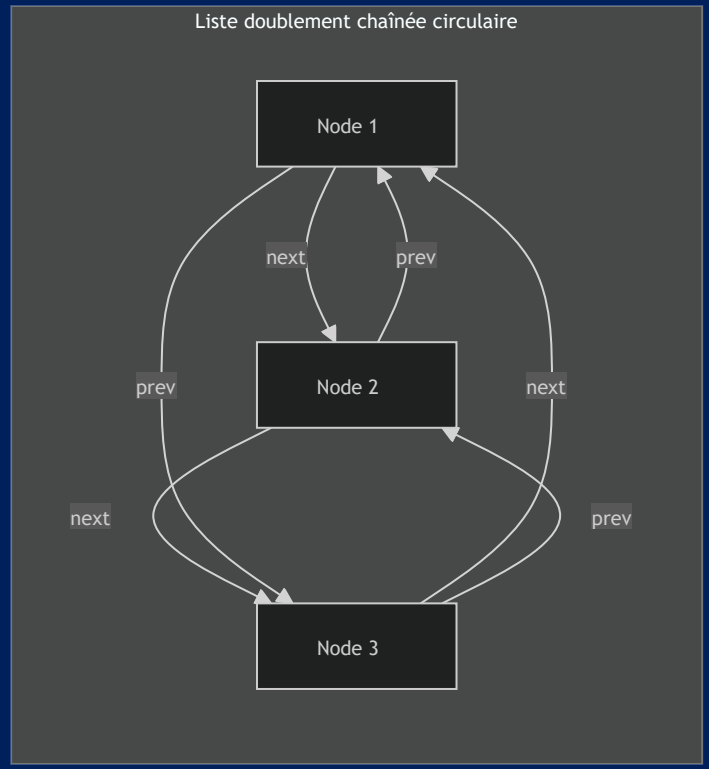

Simple liste doublement chaînée
Node 1
next
prev
Node 2
next
prev
Node 3

Listes Doublement Chaînées Circulaires

Dans cette variante :

- Le `next` du dernier nœud pointe vers le premier.
- Le `prev` du premier nœud pointe vers le dernier.

Cette structure supporte une navigation cyclique infinie dans les deux directions, idéale pour des algorithmes spécifiques (ex : gestion de files circulaires, anneaux).


Liste doublement chaînée circulaire
Node 1
next
prev
Node 2
prev
next
next
prev
Node 3

Exemples d’Opérations sur les Nœuds

Insertion après un nœud donné

```c
void inserer_apres(Node *node, int val) {
    if (!node) return;
    Node *nouveau = malloc(sizeof(Node));
    nouveau->data = val;
    nouveau->next = node->next;
    nouveau->prev = node;

    if (node->next)
        node->next->prev = nouveau;
    node->next = nouveau;
}
```

Suppression d’un nœud

```c
void supprimer(Node **head, Node *node) {
    if (!head || !*head || !node) return;

    if (node->prev)
        node->prev->next = node->next;
    else
        *head = node->next; // node est la tête

    if (node->next)
        node->next->prev = node->prev;

    free(node);
}
```

Avantages et Inconvénients

Aspects	Liste doublement chaînée
Navigation	Bidirectionnelle, accès facile dans les deux sens
Insertion/suppression	Efficace, nécessite peu de déplacements
Complexité mémoire	Coût mémoire plus élevé (deux pointeurs par nœud)
Implémentation	Plus complexe qu’une liste simple
Usage circulaire	Adaptée aux structures cycliques, gestion fluide

Ce qu’il faut retenir & Ressources

Les listes doublement chaînées et circulaires offrent une souplesse algorithmique importante, notamment pour des manipulations bidirectionnelles performantes, à la condition de bien maîtriser leur structure mémoire et le lien des pointeurs.

Sources consultées :

- GeeksforGeeks — Doubly Linked List
- TutorialsPoint — Data Structures Doubly Linked List
- Wikipedia — Doubly linked list
- Wikipedia — Circular doubly linked list

Structures de données dynamiques avancées

Listes Doublement Chaînées et Circulaires

Les listes doublement chaînées offrent une manipulation efficace des éléments grâce à un accès direct et bidirectionnel. Elles permettent des opérations d'insertion, de suppression et de parcours en tête, en queue ou à une position arbitraire, avec une complexité temporelle optimale.

Structure d'un Nœud

Un nœud dans une liste doublement chaînée contient la donnée, un pointeur vers le nœud précédent (`prev`) et un pointeur vers le nœud suivant (`next`). Un pointeur vers la tête de la liste est essentiel pour son management.

```
typedef struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
} Node;
```

Opérations d’Insertion

L’insertion d’un élément nécessite la création d’un nouveau nœud et la mise à jour des pointeurs `prev` et `next` des nœuds adjacents.

1. Insertion en tête

1. Créer un nouveau nœud.
2. Le lier à l’ancienne tête (`nouveau→next = *head`).
3. Mettre à jour le pointeur `prev` de l’ancienne tête vers le nouveau nœud.
4. Le nouveau nœud devient la nouvelle tête (`*head = nouveau`).

```
void inserer_tete(Node **head, int val) {
    Node *nouveau = malloc(sizeof(Node));
    if (!nouveau) return;
    nouveau→data = val;
    nouveau→prev = NULL;
    nouveau→next = *head;

    if (*head) (*head)→prev = nouveau;
    *head = nouveau;
}
```

2. Insertion en queue

Parcourt la liste jusqu'à la fin, puis lie le nouveau nœud au dernier, en gérant le cas de liste vide.

```c
void inserer_queue(Node **head, int val) {
    Node *nouveau = malloc(sizeof(Node));
    if (!nouveau) return;
    nouveau->data = val;
    nouveau->next = NULL;

    if (*head == NULL) {
        nouveau->prev = NULL;
        *head = nouveau;
        return;
    }

    Node *temp = *head;
    while (temp->next) temp = temp->next;

    temp->next = nouveau;
    nouveau->prev = temp;
}
```

3. Insertion à une position donnée

Parcourt la liste jusqu'à la position `pos`, puis insère le nouveau nœud **avant** le nœud à cette position, ajustant les pointeurs autour.

```c
void inserer_position(Node **head, int val, int pos) {
    if (pos == 0) {
        inserer_tete(head, val);
        return;
    }

    Node *temp = *head;
    for (int i = 0; i < pos - 1 && temp != NULL; i++)
        temp = temp->next;

    if (!temp) return; // position invalide

    Node *nouveau = malloc(sizeof(Node));
    if (!nouveau) return;
    nouveau->data = val;

    nouveau->next = temp->next;
    if (temp->next)
        temp->next->prev = nouveau;

    temp->next = nouveau;
    nouveau->prev = temp;
}
```

Opérations de Suppression

La suppression d'un élément implique la déconnexion du nœud de la liste et la libération de sa mémoire.

1. Suppression en tête

1. Vérifier que la liste n'est pas vide.
2. Le nœud à supprimer est l'actuelle tête.
3. La nouvelle tête devient le nœud suivant (`*head = temp→next`).
4. Si la nouvelle tête existe, son pointeur `prev` est mis à NULL.
5. Libérer la mémoire de l'ancien nœud tête.

```c
void supprimer_tete(Node **head) {
    if (!head || !*head) return;

    Node *temp = *head;
    *head = temp→next;

    if (*head)
        (*head)→prev = NULL;

    free(temp);
}
```

2. Suppression en queue

Parcourt la liste jusqu'au dernier nœud, déconnecte ce dernier de son précédent, puis libère sa mémoire.

```c
void supprimer_queue(Node **head) {
    if (!head || !*head) return;

    Node *temp = *head;
    if (!temp->next) { // un seul nœud
        free(temp);
        *head = NULL;
        return;
    }

    while (temp->next) temp = temp->next;

    temp->prev->next = NULL;
    free(temp);
}
```

3. Suppression à une position donnée

Parcourt la liste jusqu'à la position `pos`, déconnecte le nœud correspondant en ajustant les pointeurs `next` et `prev` de ses voisins, puis libère sa mémoire.

```c
void supprimer_position(Node **head, int pos) {
    if (!head || !*head) return;
    Node *temp = *head;

    for (int i = 0; i < pos && temp != NULL; i++)
        temp = temp->next;

    if (!temp) return;

    if (temp->prev)
        temp->prev->next = temp->next;
    else
        *head = temp->next;

    if (temp->next)
        temp->next->prev = temp->prev;

    free(temp);
}
```

Parcours de la Liste et Vue d’Ensemble

Les listes doublement chaînées peuvent être parcourues dans les deux sens, offrant une flexibilité accrue.

1. Parcours en avant

Traverse la liste de la tête à la queue via le pointeur `next`.

```
void afficher_avant(Node *head) {
    Node *temp = head;
    while (temp) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

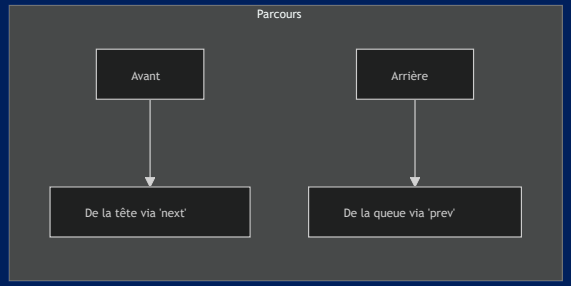
2. Parcours en arrière

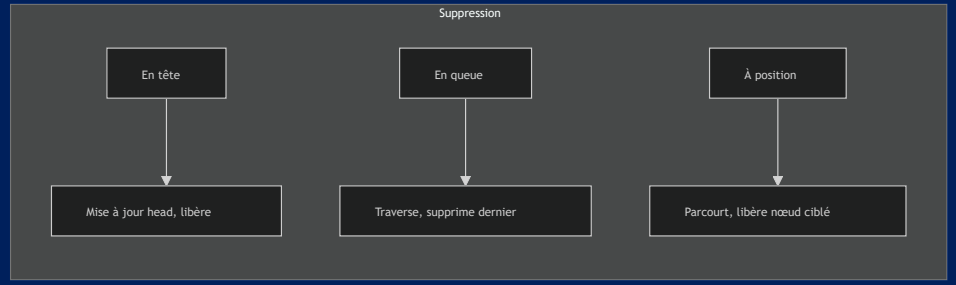
Nécessite de trouver le dernier nœud, puis de remonter vers la tête via le pointeur `prev`.

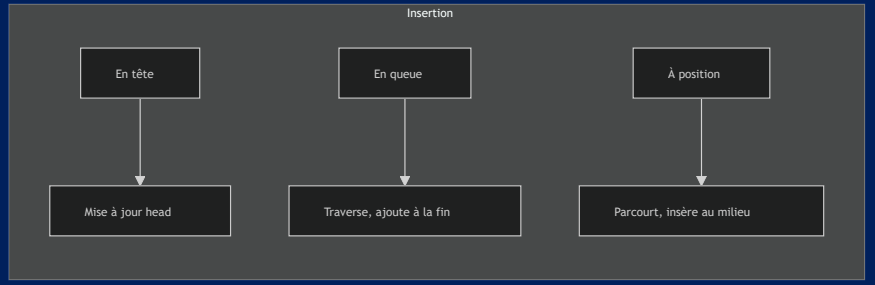
```c
void afficher_arriere(Node *head) {
    if (!head) return;
    Node *temp = head;
    while (temp->next) temp = temp->next; // Aller à la fin

    while (temp) { // Remonter
        printf("%d ", temp->data);
        temp = temp->prev;
    }
    printf("\n");
}
```

Schéma récapitulatif des opérations principales







Listes Doublement Chaînées Circulaires

Dans une liste circulaire, le pointeur `next` du dernier nœud pointe vers la tête, et le pointeur `prev` de la tête pointe vers le dernier nœud.

Détails pour les listes circulaires :

- Le test `temp→next == NULL` est remplacé par `temp→next ≠ head` pour détecter la fin.
- Toutes les opérations (insertion, suppression, parcours) doivent gérer cette "boucle infinie" pour maintenir la cohérence de la structure.

Exemple d’insertion en queue (circulaire) :

```c
void inserer_queue_circ(Node **head, int val) {
    Node *nouveau = malloc(sizeof(Node));
    nouveau->data = val;

    if (*head == NULL) { // Si la liste est vide
        nouveau->next = nouveau->prev = nouveau; // Pointe sur lui-même
        *head = nouveau;
        return;
    }

    Node *tail = (*head)->prev; // Le dernier nœud

    tail->next = nouveau;      // Le précédent dernier pointe vers le nouveau
    nouveau->prev = tail;      // Le nouveau pointe vers l'ancien dernier
    nouveau->next = *head;     // Le nouveau pointe vers la tête
    (*head)->prev = nouveau;   // La tête pointe vers le nouveau comme précédent
}
```

Ce qu’il faut retenir et Sources

Ce qu’il faut retenir :

Les opérations sur les listes doublement chaînées impliquent une gestion attentive des pointeurs `prev` et `next` pour maintenir la cohérence des liens. La maîtrise de ces opérations est indispensable pour manipuler efficacement ces structures dynamiques, aussi bien en mode linéaire que circulaire.

Sources consultées :

- GeeksforGeeks — Doubly Linked List Overview and Operations
- Programiz — Doubly Linked List in C
- TutorialsPoint — Doubly Linked List Operations
- Wikipedia — Doubly linked list

Listes Chaînées : Rappels et Fondamentaux

Introduction aux structures dynamiques

- **Liste simplement chaînée :** Chaque nœud contient une donnée et un pointeur vers le nœud *suivant*.
- **Liste doublement chaînée :** Chaque nœud contient une donnée, un pointeur vers le nœud *suivant*, et un pointeur vers le nœud *précédent*.
- **Listes circulaires :** Le dernier nœud pointe vers le premier, pouvant être simplement ou doublement chaînées.

Les Atouts des Listes Doublement Chaînées

Pourquoi choisir la bidirectionnalité ?

- **Navigation bidirectionnelle :** Le pointeur `prev` permet de parcourir la liste en avant et en arrière, facilitant la recherche inverse ou l'itération depuis la fin.
- **Suppression simplifiée :** La suppression d'un nœud (hors tête) est plus efficace car l'accès au nœud précédent est direct, sans parcours initial.
- **Insertion plus flexible :** L'insertion avant ou après un nœud est directe, même avec seulement un pointeur vers ce nœud.
- **Support des listes circulaires :** Elles facilitent le parcours continu (rotatif), idéal pour les anneaux ou buffers.
- **Base de structures avancées :** Des structures comme les déques (double-ended queues) reposent sur cette bidirectionnalité.

Inconvénients et Compromis des Listes Doublement Chaînées

Le prix de la flexibilité

Inconvénients	Explications
Coût mémoire accru	Chaque nœud stocke un pointeur supplémentaire (`prev`), augmentant la consommation mémoire.
Complexité de gestion	La gestion de `prev` et `next` est plus complexe, augmentant les risques d'erreurs.
Performances moindres	Pour un parcours avant uniquement, une liste simple est plus légère et rapide en cache.
Opérations plus coûteuses	Insertion et suppression impliquent la mise à jour de deux pointeurs.

Comparaison Rapide : Simple vs. Doublement Chaînée

Un choix stratégique selon les besoins

Critère	Liste simplement chaînée	Liste doublement chaînée
Stockage	1 pointeur par nœud (next)	2 pointeurs par nœud (prev + next)
Navigation	Unidirectionnelle	Bidirectionnelle
Suppression	Nœud précédent non accessible	Nœud précédent accessible directement
Insertion	Pas d’accès facilité avant un nœud	Accès flexible avant et après un nœud
Complexité code	Plus simple	Plus complexe
Usage mémoire	Moins	Plus
Adaptation Algorithmes	Moins adaptée (ex: parcours inverse)	Plus adaptée (ex: déques, listes circulaires)

Cas Pratique : Efficacité de la Suppression

Suppression d'un nœud donné (hors tête) dans une liste simplement chaînée :

- Nécessite de parcourir la liste depuis le début pour trouver le nœud précédent.
- Coût en temps : (O(n)).

```c
void supprimerNode(Node **head, Node *node) {
    if (*head == node) {
        *head = (*head)->next;
        free(node);
        return;
    }

    Node *temp = *head;
    while (temp && temp->next != node) {
        temp = temp->next;
    }

    if (temp == NULL) return;

    temp->next = node->next;
    free(node);
}
```

Dans une liste doublement chaînée :

- Accès direct au nœud précédent via `node→prev`.
- Pas de parcours nécessaire.
- Coût en temps : (O(1)).

```c
void supprimerNodeD(Node **head, Node *node) {
    if (node→prev)
        node→prev→next = node→next; // Le nœud précédent pointe vers le suivant
    else
        *head = node→next; // Le nœud était la tête

    if (node→next)
        node→next→prev = node→prev; // Le nœud suivant pointe vers le précédent

    free(node);
}
```

Décision Stratégique & Ressources Complémentaires

Ce qu’il faut retenir :

La sélection entre liste simplement ou doublement chaînée dépend des exigences fonctionnelles : la bidirectionnalité apporte plus de flexibilité et efficacité dans certains cas, au prix d’une surcharge mémoire et d’une gestion de pointeurs plus minutieuse.

Sources consultées :

- GeeksforGeeks — Difference between Singly and Doubly Linked List
- TutorialsPoint — Singly vs Doubly Linked Lists
- Wikipedia — Doubly linked list
- Programiz — Doubly Linked List

Listes Doublement Chaînées et Circulaires

- Une **liste circulaire** est une structure de données dynamique où le dernier nœud pointe vers le premier, formant un cycle.
- Combinée à une structure doublement chaînée, elle permet un parcours bidirectionnel infini.
- **Usages courants** : files circulaires, gestion d’anneaux, algorithmes de buffer.

Variantes de Listes Circulaires

Simplement Chaînées vs Doublement Chaînées

Type	Description	Illustration des liens
Liste simplement chaînée circulaire	Le dernier nœud pointe vers la tête. Permet un seul sens de parcours circulaire.	`Last → First → ... → Last`
Liste doublement chaînée circulaire	Le dernier pointe vers la tête et la tête pointe vers le dernier via `prev`. Navigation dans les deux sens en boucle fermée.	`Last ⟷ First ⟷ ... ⟷ Last`

La Structure d’un Nœud

Pour une Liste Doublement Chaînée Circulaire

```
typedef struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
} Node;
```

- Dans ce modèle, la tête (`head`) de la liste est un pointeur vers le premier nœud.
- Le nœud dernier a `next` pointant vers la tête.
- La tête a `prev` pointant vers le dernier.

Implémentation : Opérations d’Insertion

Insertion en Tête :

```c
void inserer_tete_circulaire(Node **head, int val) {
    Node *nouveau = malloc(sizeof(Node));
    if (!nouveau) return;
    nouveau->data = val;

    if (*head == NULL) {
        nouveau->next = nouveau->prev = nouveau;
        *head = nouveau;
        return;
    }

    Node *last = (*head)->prev;

    nouveau->next = *head;
    nouveau->prev = last;

    last->next = nouveau;
    (*head)->prev = nouveau;

    *head = nouveau;
}
```

Insertion en Queue :

```c
void inserer_queue_circulaire(Node **head, int val) {
    if (*head == NULL) {
        inserer_tete_circulaire(head, val);
        return;
    }

    Node *last = (*head)->prev;
    Node *nouveau = malloc(sizeof(Node));
    if (!nouveau) return;

    nouveau->data = val;
    nouveau->next = *head;
    nouveau->prev = last;

    last->next = nouveau;
    (*head)->prev = nouveau;
}
```

Gérer et Naviguer dans les Éléments

Suppression d'un Nœud Donné :

```c
void supprimer_node_circulaire(Node **head, Node *node) {
    if (!head || !*head || !node) return;

    if (node->next == node && node->prev == node) { // Un seul nœud
        free(node);
        *head = NULL;
        return;
    }

    node->prev->next = node->next;
    node->next->prev = node->prev;

    if (*head == node)
        *head = node->next;

    free(node);
}
```

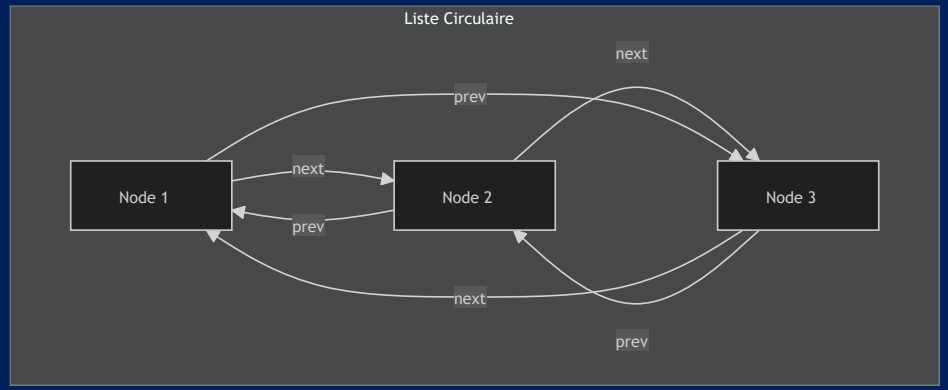
Parcours de la Liste Circulaire (avant) :

```c
void afficher_liste_circulaire(Node *head) {
    if (!head) return;
    Node *temp = head;

    do {
        printf("%d ", temp->data);
        temp = temp->next;
    } while (temp != head);

    printf("\n");
}
```

Structure Visuelle



Visualisation, Particularités & Ressources

Ce qu’il faut retenir

- La liste peut être vide (`head == NULL`).
- Les cas spéciaux (un seul nœud) doivent être correctement gérés.
- Le pointeur `head` reste utile pour toujours accéder au début, bien que la liste soit cyclique.
- La liste doublement chaînée circulaire combine les avantages du parcours bidirectionnel avec la flexibilité d’une structure sans fin, essentielle pour des algorithmes nécessitant un contrôle fluide des rotations et accès cycliques.

Sources consultées

- GeeksforGeeks — Circular Doubly Linked List
- Programiz — Circular Doubly Linked List in C
- TutorialsPoint — Circular Doubly Linked List Algorithms
- Wikipedia — Circular doubly linked list

Tables de Hachage

La table de hachage : Accès rapide aux données

- Structure de données dynamique permettant d'associer **clefs et valeurs**.
- Offre un accès proche de **(O(1)) en moyenne**.
- Repose sur une **fonction de hachage** qui transforme une clef en un **indice** dans un tableau.

Fonctionnement résumé :

1. Calcul d'un **hachage** (entier) de la clé.
2. Cet entier sert d'**indice** pour stocker ou récupérer la valeur.
3. Gestion des **collisions** (plusieurs clefs même hachage) par des techniques spécifiques.

1. Principe du Hachage & 2. Fonction de Hachage

Principe du hachage :

- Transforme une clef en un indice unique (si possible) pour un accès direct.
- Nécessite une gestion efficace des cas où différentes clefs génèrent le même indice (collisions).

Fonction de hachage :

- Prend une clef (k) et renvoie un indice dans une table de taille (m).
- **Propriétés recherchées :**
  - **Uniformité** : Répartition homogène des indices pour minimiser les collisions.
  - **Rapidité** : Calcul efficace et rapide de l'indice.
  - **Déterminisme** : Même clef doit toujours retourner la même valeur.
  - **Sensibilité aux différences** : Clefs légèrement différentes doivent produire des hachages distincts.

3. Exemples de fonctions de hachage

Pour des entiers :

- Une fonction simple et courante : [h(k) = k \mod m] où (m) est la taille de la table (souvent un nombre premier pour une meilleure répartition).

Pour des chaînes de caractères : Le hash DJB2

- Se base sur un calcul cumulatif pondéré des caractères.
- Exemple de principe (DJB2) :

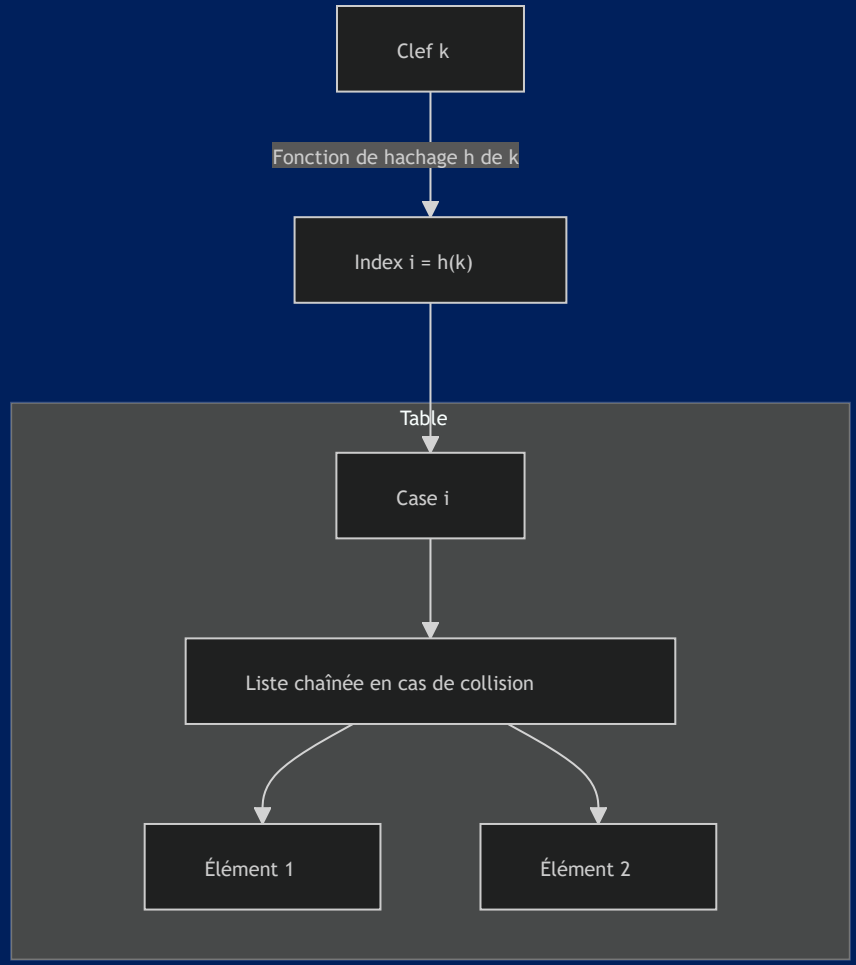
```c
unsigned long djb2(const char *str) {
    unsigned long hash = 5381; // Valeur initiale
    int c;
    while ((c = *str++))
        hash = ((hash << 5) + hash) + c; /* hash * 33 + c */
    return hash;
}
```

- Cette fonction est simple, rapide et offre une bonne uniformité en pratique.

4. Gestion des collisions

Les collisions sont inévitables et doivent être gérées efficacement pour maintenir les performances (O(1)).

- **Chaînage** :
  - Chaque case de la table contient une liste (chaînée) d'éléments.
  - En cas de collision, l'élément est ajouté à la liste correspondant à l'indice.
- **Sondage (Probing)** :
  - En cas de collision, on cherche une autre case disponible.
  - Exemples : sondage linéaire, quadratique, double hachage.


Clef k
Fonction de hachage h de k
Index i = h(k)
Table
Case i
Liste chaînée en cas de collision
Élément 1
Élément 2

Exemple d’utilisation simple d’une table de hachage avec chaînage

```c
#define TABLE_SIZE 101 // Taille de la table

typedef struct Node {
    char *key;
    int value;
    struct Node *next;
} Node;

Node *hash_table[TABLE_SIZE]; // Le tableau des "cases" (têtes de listes)

// Fonction de hachage (DJB2 simplifié modulo TABLE_SIZE)
unsigned int hash(char *key) {
    unsigned long hash_val = 5381;
    int c;
    while ((c = *key++))
        hash_val = ((hash_val << 5) + hash_val) + c;
    return hash_val % TABLE_SIZE; // Retourne l'indice dans la table
}
// Insertion simple avec chaînage
void insert(char *key, int value) {
    unsigned int idx = hash(key); // Calcul de l'indice
    Node *new_node = malloc(sizeof(Node)); // Allocation d'un nouveau nœud
    new_node->key = strdup(key);
    new_node->value = value;
    new_node->next = hash_table[idx]; // Le nouveau nœud pointe vers l'ancienne tête de liste
    hash_table[idx] = new_node;       // Le nouveau nœud devient la tête de liste
}
```

Limites et considérations & Ce qu’il faut retenir

Limites importantes :

- Une **fonction de hachage mal conçue** génère de nombreuses collisions, dégradant les performances.
- La **taille de la table** est cruciale (facteur de charge (\alpha = \frac{n}{m})) et doit être **redimensionnée** en cas de surcharge.
- La **synchronisation** est nécessaire en contexte multithread pour éviter les incohérences.

Ce qu’il faut retenir :

- Le hachage et les fonctions de hachage sont des piliers pour un accès rapide aux données.
- Ils transforment les clés en indices mémoire de manière calculable et reproductible.

8. Sources consultées :

- GeeksforGeeks — Hashing Data Structure
- Wikipedia — Hash function
- Wikipedia — Hash table
- DJB2 Hash Function Explanation (Stack Overflow)
- Programiz — Hash Table and Hash Functions

Le Défi des Collisions

Une **collision** survient quand deux clefs différentes produisent la même valeur de hachage, pointant vers un même indice de la table.

La gestion efficace de ces collisions est **essentielle pour préserver la rapidité d’accès** des tables de hachage.

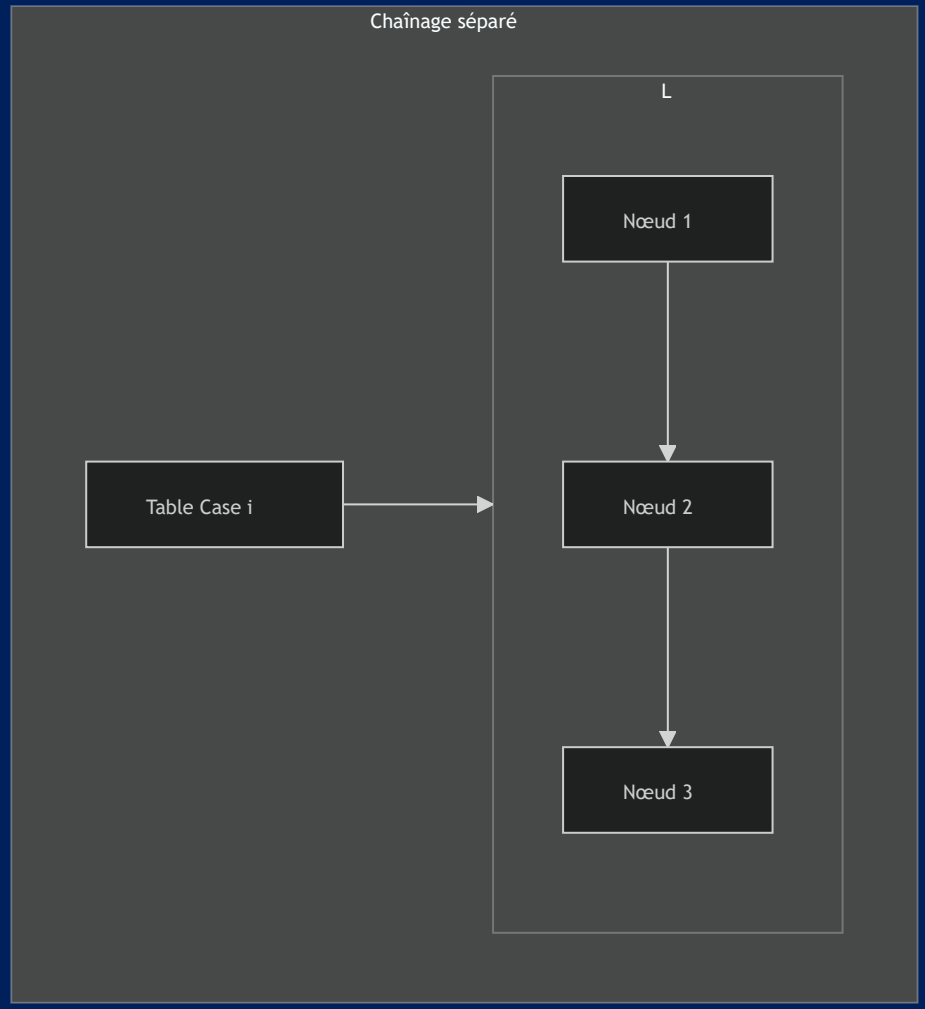
Méthode 1 : Le Chaînage séparé

Principe

Chaque case de la table contient une **liste chaînée** d'éléments. En cas de collision, les nouveaux éléments sont ajoutés à la liste de la case concernée.

Avantages

- **Facile à implémenter**.
- **Pratique** en cas de facteur de charge > 1 (plus d'éléments que de cases).
- **Performant** même dans le pire des cas (la recherche traverse uniquement la liste).
- **Complexité moyenne :** (O(1 + \alpha)), où (\alpha) est le facteur de charge.


Chaînage séparé
L
Nœud 1
Table Case i
Nœud 2
Nœud 3

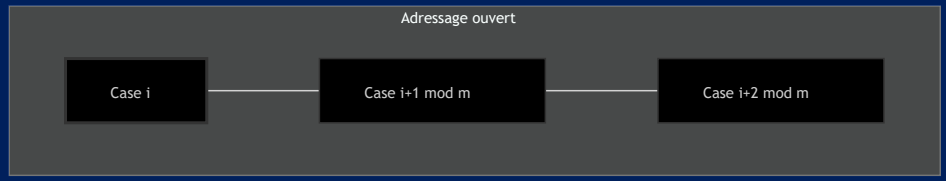
3. Méthode 2 : L’Adressage Ouvert

Principe

Les éléments sont stockés **directement dans la table**, sans listes externes. En cas de collision, on recherche une autre case "libre" selon une stratégie de **sondage**.

Stratégies principales :

- **Sondage linéaire**
- **Sondage quadratique**
- **Double hachage**



Adressage Ouvert : Les Stratégies de Sondage

1 Sondage linéaire

- Cherche la première case suivante vide (à l'indice `(h(k)+i) mod m`).
- **Inconvénient :** Peut conduire à des regroupements dits "**clustering primaire**", dégradant les performances.

```
int linear_probing_insert(int table[], int m, int key) {
    int idx = hash(key);
    int i = 0;
    while (table[(idx + i) % m] ≠ EMPTY) {
        i++;
    }
    table[(idx + i) % m] = key;
    return (idx + i) % m;
}
```

2 Sondage quadratique

- Cherche la prochaine case testée selon `(h(k) + i^2) mod m`.
- **Avantage :** Réduit le clustering primaire.
- **Inconvénient :** Peut générer du "**clustering secondaire**".

```c
int quadratic_probing_insert(int table[], int m, int key) {
    int idx = hash(key, m);
    int i = 0;

    while (table[(idx + i * i) % m] != EMPTY) {
        i++;
        if (i == m) { // table pleine
            printf("Table pleine, impossible d'insérer %d\n", key);
            return -1;
        }
    }

    table[(idx + i * i) % m] = key;
    return (idx + i * i) % m;
}
```

3 Double hachage

- Utilise une deuxième fonction de hachage (h_2(k)) pour décaler plus efficacement en cas de collision.

Le principe :

- On calcule deux fonctions de hashage :
- -h1(key) → position de départ.
- -h2(key) → pas de déplacement en cas de collision.
- Si la case est occupée, on essaie :

(h1(key)+i×h2(key)) mod m

où i est le nombre d’essais (0, 1, 2…).

Chaînage vs Adressage Ouvert

Méthode	Mémoire supplémentaire	Complexité insertion	Gestion facteur de charge élevé	Implementation
Chaînage séparé	Oui (listes externes)	(O(1 + \alpha))	Très bonne	Simple
Adressage linéaire	Non	(O(1)) (moyenne)	Réduit efficacité au-delà de 0.7	Simple
Adressage quadratique	Non	Plus élevé que linéaire	Meilleur que linéaire	Moyennement simple
Double hachage	Non	Très bonne	Très bonne	Plus complexe

Ce qu’il faut retenir & Références

Choix et limites

- Le chaînage séparé facilite les **redimensionnements dynamiques**.
- L’adressage ouvert demande un **facteur de charge < 1** pour éviter les performances dégradées.
- Le choix dépend du **contexte** : mémoire disponible, complexité, dimensionnement, type de clés.

La gestion des collisions est la pierre angulaire des tables de hachage robustes. En fonction des contraintes mémoire et d’efficacité, chaînage ou adressage ouvert offrent des compromis adaptés.

Sources consultées

- GeeksforGeeks — Collision Resolution Techniques
- Wikipedia — Hash table
- Programiz — Hash Table Collision Handling
- Open Addressing with Linear, Quadratic, Double Hashing

Tables de Hachage : Complexité des Opérations

Les tables de hachage offrent un **accès rapide en moyenne**, proche de (O(1)). Cependant, la complexité dépend fortement de deux facteurs essentiels :

1. Le comportement de la **fonction de hachage**.
2. La **gestion des collisions**.

Le Facteur de Charge (α)

Si la table a m = 10 cases et qu'on a inséré n = 7 éléments :

α=0.7

Cela signifie que la table est remplie à 70 %.

c'est le rapport entre le nombre d'éléments (n) stockés et la taille (m) de la table. Il indique la densité d'occupation.

- **(α) faible (ex : 0.5)** : Performances optimales.
- **(α) proche de 1 ou plus** : Dégradation des performances, surtout avec l'adressage ouvert.

Chaînage Séparé : Complexité des Opérations

Cette méthode de résolution des collisions stocke les éléments en listes chaînées pour chaque indice de table.

Opération	Complexité moyenne	Complexité pire cas
Insertion	(O(1))	(O(n)) (tous éléments dans une liste)
Recherche	(O(1 + α))	(O(n))
Suppression	(O(1 + α))	(O(n))

Explication

Les opérations accèdent à la liste chaînée correspondant à l'indice calculé. La complexité moyenne est linéaire en (α), tandis que le pire cas est linéaire en (n) si la fonction de hachage est très mauvaise (tous les éléments dans une seule liste).

Adressage Ouvert : Complexité des Opérations

En adressage ouvert, toutes les entrées sont stockées directement dans la table, et les collisions sont gérées par des séquences de sondage.

Opération	Complexité
Insertion	$O(1 / 1-\alpha)$
Recherche	$O(1 / 1-\alpha)$
Suppression	$O(1 / 1-\alpha)$

Explication

- Plus le facteur de charge (α) se rapproche de 1, plus le temps moyen d'exécution augmente de façon significative.
- Les méthodes de sondage (linéaire, quadratique, double hachage) ont un impact direct sur ces performances.

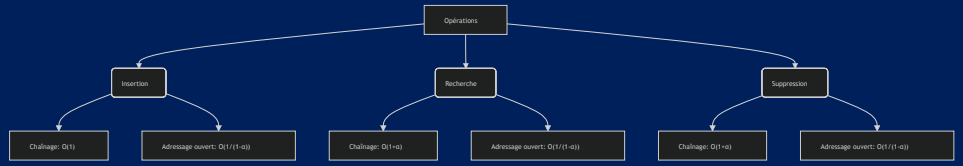
Exemples concrets

Recherche en chaînage Dans une table de taille 10 avec 5 éléments ((\alpha=0.5)), une recherche moyenne inspecte environ (1 + 0.5 = 1.5) éléments par case.

Insertion en adressage linéaire Si (\alpha=0.8), le nombre moyen de sondages avant insertion est d’environ (\frac{1}{1-0.8} = 5). Cela représente 5 accès mémoire en moyenne avant l’insertion.

Synthèse Opérationnelle de la Fonction de Hachage

Complexité des opérations par type de gestion des collisions



L’Impact du choix de la fonction de hachage

- Une **fonction de hachage bien conçue** limite le nombre de collisions, réduisant ainsi le (\alpha) effectif et optimisant les performances moyennes.
- Un **mauvais hachage** concentre les éléments dans quelques cases, dégradant la complexité moyenne vers le pire cas (O(n)).

Clés de la Performance et Ressources

Ce qu’il faut retenir

La réalité d’une table de hachage performante repose sur :

1. Le **contrôle du facteur de charge** (\alpha).
2. La **qualité de la fonction de hachage**.

Ces deux paramètres déterminent directement le coût en temps moyen et en pire cas des opérations d’insertion, recherche et suppression.

Sources consultées

- GeeksforGeeks — Time Complexity of Hashing
- Wikipedia — Hash Table#Complexity_of_hash_tables
- Programiz — Hash Table Analysis
- MIT Lecture Notes — Hashing Performance

Section 3 : Arbres Binaires de Recherche (ABR)

1. Qu’est-ce qu’un Arbre Binaire de Recherche (ABR) ?

Un **Arbre Binaire de Recherche (BST)** est une structure de données arborescente respectant des règles précises :

- Chaque nœud contient une **clé** (valeur).
- Pour tout nœud (N):
  - Toutes les clés du **sous-arbre gauche** sont **strictement inférieures** à la clé de (N).
  - Toutes les clés du **sous-arbre droit** sont **supérieures** à la clé de (N).
- Chaque nœud a **au plus deux enfants** (gauche et droit).

2. Propriétés Fondamentales et Leurs Conséquences

Les caractéristiques d'un ABR déterminent son comportement et son efficacité.

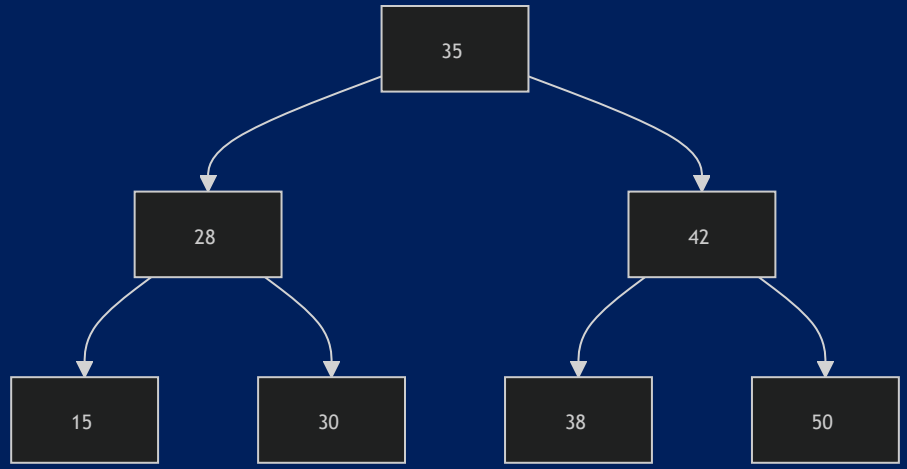
Propriété	Description
Recherche efficace	Opérations (recherche, insertion, suppression) en (O(h)) en moyenne, où (h) est la hauteur.
Ordre des clés	Le parcours infixe retourne les clés dans l'ordre croissant.
Hauteur variable	De (\log n) (équilibré) à (n) (dégénéré) pour (n) nœuds, selon l'insertion.
Structure récursive	Chaque sous-arbre est lui-même un ABR.

Conséquences :

- Algorithmes récursifs simples pour les opérations de base.
- La **complexité dépend fortement de la hauteur** de l'arbre.
- Les arbres non équilibrés dégradent les performances (temps linéaire, comme une liste chaînée).

3. Exemple Illustratif d’un ABR

Visualisons la construction d’un ABR après l’insertion successive des clés : **35, 28, 42, 15, 30, 38, 50**.



Ce diagramme montre comment les clés sont organisées pour respecter la propriété d’ordre des ABR.

4. Algorithme de Recherche dans un ABR

La recherche d'une clé dans un ABR tire parti de sa structure ordonnée, permettant une exploration rapide.

```
Node* rechercher(Node *root, int key) {
    // Cas de base : arbre vide ou clé trouvée
    if (root == NULL || root->key == key)
        return root;

    // Si la clé est plus petite, chercher dans le sous-arbre gauche
    if (key < root->key)
        return rechercher(root->left, key);
    // Sinon, chercher dans le sous-arbre droit
    else
        return rechercher(root->right, key);
}
```

L'algorithme parcourt l'arbre de manière récursive, se dirigeant à gauche ou à droite en fonction de la comparaison avec la clé du nœud courant.

5. Parcours Infixe : Obtenir les Clés Triées

Le parcours infixe est une méthode fondamentale pour visiter les nœuds d'un ABR. Sa particularité est de retourner les clés dans un ordre croissant.

```c
void parcours_infixe(Node *root) {
    if (root ≠ NULL) {
        // 1. Visiter le sous-arbre gauche
        parcours_infixe(root→left);
        // 2. Traiter le nœud courant
        printf("%d ", root→key);
        // 3. Visiter le sous-arbre droit
        parcours_infixe(root→right);
    }
}
```

Ce parcours est essentiel pour vérifier l'ordre des clés ou pour extraire une liste triée des éléments de l'arbre.

6. Points Cruciaux & Conclusion

Points à surveiller :

- L'**ordre d'insertion** détermine la forme de l'arbre et donc sa hauteur.
- L'**équilibrage** est indispensable pour garantir une complexité (O(\log n)) dans le pire cas (cf. arbres AVL, rouges-noirs).
- La **suppression** d'un nœud est complexe et doit gérer plusieurs cas (feuille, 1 enfant, 2 enfants).

Conclusion : Un ABR est une base essentielle en algorithmique servant de fondation pour des structures plus complexes. Son organisation garantit un accès ordonné efficace dès lors que sa hauteur est maîtrisée.

Sources consultées :

- GeeksforGeeks — Binary Search Tree (BST)
- Wikipedia — Binary Search Tree
- TutorialsPoint — Binary Search Tree
- Programiz — Binary Search Tree

Arbres Binaires de Recherche (ABR) : Les Fondamentaux

- **Qu’est-ce qu’un ABR ?**
  - Une structure de données conçue pour maintenir l’ordre.
  - Permet des opérations efficaces sur les données.
- **Les 3 opérations fondamentales :**
  1. Recherche
  2. Insertion
  3. Suppression
- **Propriété clé :** Chaque nœud sépare les clés plus petites (à gauche) et les plus grandes (à droite).

Recherche d’un Élément dans un ABR

L’opération de recherche exploite la propriété de l’ABR.

- **Algorithme (récursif) :**
  - Si l’arbre est vide, l’élément n’est pas trouvé.
  - Si la clé recherchée est celle du nœud courant, retourner le nœud.
  - Si la clé est < clé du nœud, rechercher dans le sous-arbre gauche.
  - Sinon, rechercher dans le sous-arbre droit.

- **Exemple de code © :**

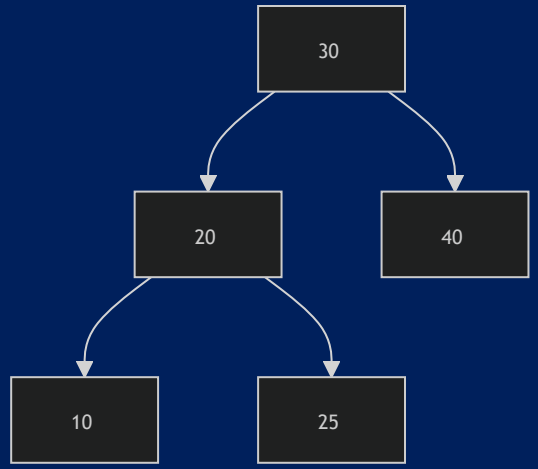
```c
Node* recherche(Node *root, int key) {
    if (root == NULL || root->key == key)
        return root;
    if (key < root->key)
        return recherche(root->left, key);
    else
        return recherche(root->right, key);
}
```

- **Complexité :**
  - Moyenne : (O(\log n)) (si l'arbre est équilibré)
  - Pire cas : (O(n)) (si l'arbre est dégénéré, comme une liste)

Insertion d'un Nouvel Élément dans un ABR

L'insertion respecte la structure de l'ABR en plaçant la nouvelle clé à une feuille.

- **Algorithme :**
  - Si l'arbre est vide, créer un nouveau nœud.
  - Comparer la clé avec la racine.
  - Insérer récursivement à gauche si la clé < racine, sinon à droite.
- **Illustration d'une insertion (ex: Insérer `25`) :**

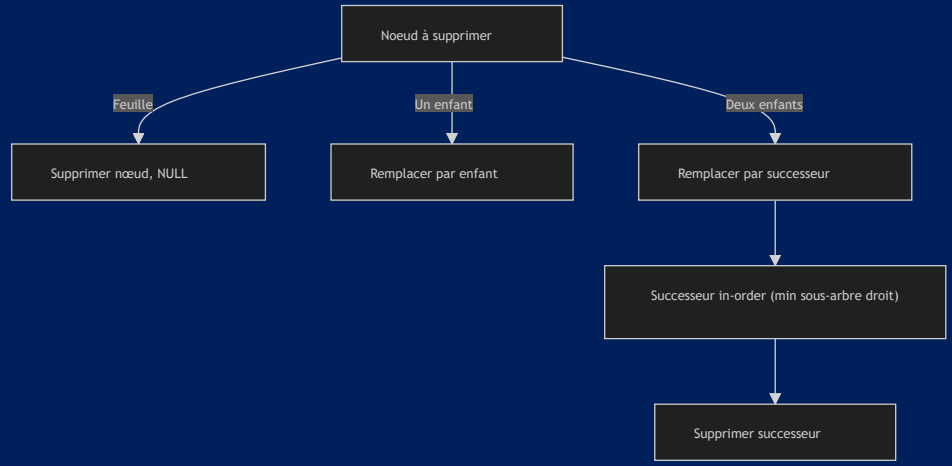


Suppression d’un Élément dans un ABR : Gestion des Cas

La suppression est l’opération la plus délicate, avec trois cas principaux.

- **Cas 1 : Le nœud est une feuille (0 enfant)**
  - Supprimer simplement le nœud ; le parent pointe vers `NULL`.
- **Cas 2 : Le nœud a un seul enfant**
  - Le parent relie directement cet enfant, "sautant" le nœud supprimé.
- **Cas 3 : Le nœud a deux enfants**
  - Trouver le successeur in-order (le plus petit nœud du sous-arbre droit).
  - Remplacer la clé du nœud supprimé par la clé de ce successeur.
  - Supprimer récursivement le successeur.

- **Illustration des cas de suppression :**



Synthèse des Complexités sur un ABR

La complexité des opérations dépend grandement de l’équilibre de l’arbre.

Opération	Temps moyen	Pire cas
Recherche	$O(\log n)$	$O(n)$
Insertion	$O(\log n)$	$O(n)$
Suppression	$O(\log n)$	$O(n)$

- **Remarque :** Les temps moyens sont atteints lorsque l’arbre est relativement équilibré. Dans le pire cas (arbre dégénéré), les performances chutent à celles d’une liste chaînée.

Points Clés & Ressources Utiles

- **Ce qu’il faut retenir :**
  - Les opérations sur un ABR nécessitent une bonne compréhension des différents cas, notamment pour la suppression.
  - La maîtrise de ces algorithmes garantit une manipulation efficace et cohérente des données structurées.
- **Sources consultées :**
  - GeeksforGeeks — BST operations
  - Wikipedia — Binary search tree#Deletion
  - Programiz — BST operations
  - TutorialsPoint — BST Deletion

Arbres Binaires de Recherche (ABR) : Les Parcours

Introduction aux parcours d'arbres Un parcours d'arbre consiste à visiter tous les nœuds selon un ordre donné. Pour les arbres binaires, il existe principalement trois parcours classiques :

- **Infixe (In-order)**
- **Préfixe (Pre-order)**
- **Postfixe (Post-order)**

Chacun a ses usages et propriétés spécifiques.

Parcours Infixe (In-order)

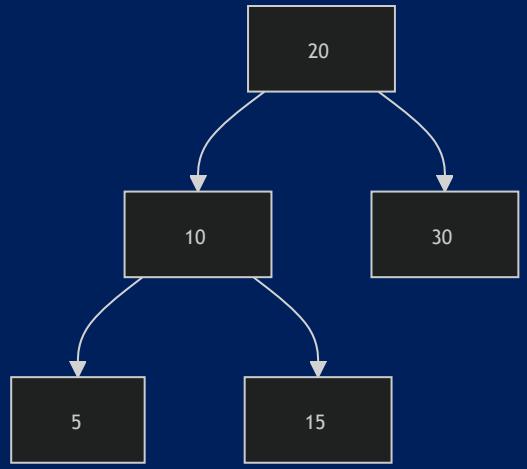
Définition Visite le sous-arbre gauche, puis le nœud courant, puis le sous-arbre droit.

Algorithme récursif

```
void inOrder(Node *root) {
    if (root ≠ NULL) {
        inOrder(root→left);
        printf("%d ", root→key);
        inOrder(root→right);
    }
}
```

Propriété importante Pour un **ABR**, le parcours infixe restitue les clés dans l'ordre **croissant**.

Exemple



Parcours infixe : `5 10 15 20 30`

Parcours Préfixe (Pre-order)

Définition Visite le nœud courant, puis le sous-arbre gauche, enfin le sous-arbre droit.

Algorithme récursif

```c
void preOrder(Node *root) {
    if (root ≠ NULL) {
        printf("%d ", root→key);
        preOrder(root→left);
        preOrder(root→right);
    }
}
```

Applications

- Sauvegarde ou clonage d'un arbre (recrée l'arbre dans le même ordre).
- Évaluation d'expressions en notation préfixe (notation polonaise).

Parcours Postfixe (Post-order)

Définition Visite le sous-arbre gauche, le sous-arbre droit, puis le nœud courant.

Algorithme récursif

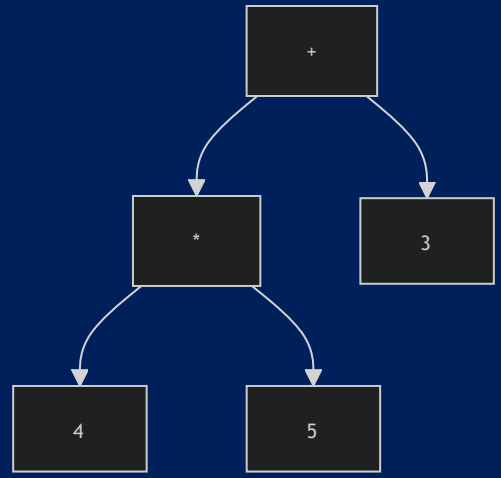
```c
void postOrder(Node *root) {
    if (root ≠ NULL) {
        postOrder(root→left);
        postOrder(root→right);
        printf("%d ", root→key);
    }
}
```

Applications

- Suppression ou libération d'un arbre (libération des fils avant le parent).
- Évaluation d'expressions en notation postfixe (notation polonaise inverse).

Synthèse Visuelle et Exemple Concret

Exemple : Expression arithmétique ((3 + (4 \times 5)))



- Parcours préfixe : `+ 3 * 4 5`
- Parcours infixe : `3 + 4 * 5` (respecte la priorité des opérateurs)
- Parcours postfixe : `3 4 5 * +`

Bilan : Complexité, Usages Clés et Références

Complexité Tous les parcours visitent chaque nœud exactement une fois, d'où une complexité linéaire : [O(n)]

Applications concrètes (Synthèse)

Parcours	Usage principal
Infixe	Récupérer les données triées dans un ABR
Préfixe	Copier un arbre, générer une expression préfixe
Postfixe	Libération mémoire, évaluation expression postfixe

Ce qu'il faut retenir Les parcours d'arbres sont des outils indispensables pour exploiter la structure des ABR, qu'il s'agisse d'accéder aux données triées, d'opérer sur des expressions ou de gérer la mémoire.

Sources consultées

- GeeksforGeeks — Tree traversal
- Wikipedia — Tree traversal
- Programiz — Tree Traversal
- Visualgo — Tree traversals

La Hauteur de l’ABR : Clé de l’Efficacité

- L’efficacité des opérations (recherche, insertion, suppression) sur un ABR dépend directement de sa **hauteur (h)**.
- **Définition :** La hauteur (h) est la longueur maximale du chemin de la racine vers une feuille.
- Les opérations parcourent au maximum un chemin racine-feuille, d’où leur dépendance à (h).
- **Objectif :** Obtenir (h = O(\log n)) pour un arbre équilibré ((n) nœuds).
- **Risque :** Sans contrôle, l’arbre peut dégénérer en liste chaînée, menant à (h = O(n)).

Complexité des Opérations Standard sur ABR

Opération	Complexité moyenne	Pire cas
Recherche	$O(\log n)$	$O(n)$
Insertion	$O(\log n)$	$O(n)$
Suppression	$O(\log n)$	$O(n)$

Explications :

- **Recherche :** Implique une comparaison successive des clés en descendant, sur une hauteur maximale (h).
- **Insertion :** Suit la logique de la recherche pour identifier le point d'ajout.
- **Suppression :** Nécessite une recherche préalable, puis un ajustement des liens, potentiellement complexe si le nœud a deux enfants.

L’Impact de l’Ordre d’Insertion sur la Hauteur

L’ordre dans lequel les éléments sont insérés influence drastiquement la structure de l’ABR et donc sa hauteur :

- **Cas optimal : Arbre équilibré**

  (Éléments insérés dans un ordre qui équilibre l’arbre)

  Séquence : (1, 5, 3, 7, 2, 6, 4)

  Hauteur : (\approx \log_2 n)

- **Cas pire : Arbre dégénéré**

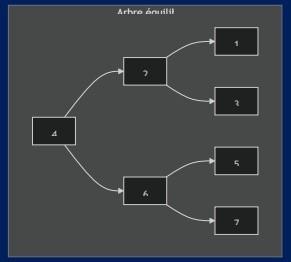
  (Éléments insérés en ordre croissant ou décroissant)

  Séquence : (1, 2, 3, 4, 5, 6, 7)

  Hauteur : (n) (l’arbre se transforme en une liste chaînée)

Visualisation : ABR Équilibré vs. Dégénéré





Synthèse : L'Équilibrage est Essentiel

Pourquoi l'équilibrage est crucial ?

- Une hauteur en (\Theta(\log n)) garantit une complexité algorithmique optimale.
- Les ABR non équilibrés entraînent des coûts linéaires ((O(n))), dégradant les performances.
- D'où l'importance des arbres auto-équilibrés (ex: AVL, arbres rouges-noirs) pour maintenir une hauteur logarithmique.

Comparaison avec d'autres structures :

Structure	Recherche	Insertion	Suppression
ABR non équilibré	(O(n))	(O(n))	(O(n))
ABR équilibré (AVL)	(O(\log n))	(O(\log n))	(O(\log n))
Table de hachage	(O(1)) (moyenne)	(O(1))	(O(1))

Ce qu'il faut retenir : Pour exploiter pleinement le potentiel d'un ABR, il est indispensable de comprendre que la hauteur conditionne la complexité des opérations. La gestion efficace de l'équilibre est la clé pour maintenir un comportement performant.

Sources consultées :

- GeeksforGeeks — BST Operations Time Complexity
- Wikipedia — Binary Search Tree#Time_complexity
- Programiz — Binary Search Tree

Limites des ABR et quel est le problème ?

Un **Arbre Binaire de Recherche (ABR)** organise les données pour des opérations rapides (recherche, insertion, suppression). Sa performance dépend crucialement de sa **hauteur**.

Le problème survient avec les **ABR dégénérés** :

- La structure s'apparente à une **liste chaînée**.
- Chaque nœud n'a qu'**un seul enfant**.
- La hauteur (h) devient linéaire par rapport au nombre de nœuds (n), soit (h=O(n)).

Comprendre la Dégénérescence

Un ABR dégénéré perd ses avantages car ses opérations deviennent aussi coûteuses que dans une liste chaînée.

Caractéristiques clés :

- Chaque nœud ne possède qu'un seul enfant (à gauche ou à droite).
- La hauteur de l'arbre est proportionnelle au nombre de nœuds ((h=O(n))).

Origine fréquente : Ce cas se produit souvent lors de l'insertion de clés dans un ordre **strictement croissant ou décroissant** sans mécanisme de rééquilibrage.

Impact sur les Performances : ABR Dégénérés vs. Équilibrés

La dégénérescence dégrade la complexité des opérations, transformant un arbre performant en une structure inefficace.

Aspect	ABR équilibré	ABR dégénéré
Hauteur (h)	$O(\log n)$	$O(n)$
Complexité recherche	$O(\log n)$	$O(n)$
Complexité insertion	$O(\log n)$	$O(n)$
Complexité suppression	$O(\log n)$	$O(n)$

Visualiser la Dégénérescence

Insertion dans l’ordre croissant (dégénéré)

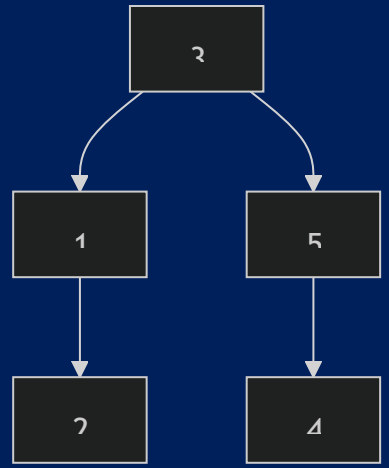
Insérer 1, 2, 3, 4, 5 :



La hauteur est égale au nombre de nœuds.

Insertion dans un ordre "équilibré"

Insérer 3, 1, 5, 2, 4 :



Structure plus plate, chemins d’accès plus courts.

Origines du Problème et Nécessité d’une Solution

La dégénérescence est le résultat direct de l’absence de stratégies de rééquilibrage.

Causes principales :

- **Insertions triées** (croissantes ou décroissantes).
- **Suppressions non optimisées** sans ajustement de la structure.

Cette faiblesse fondamentale des ABR simples motive la conception de structures plus robustes.

Arbres Équilibrés : La Solution

Pour éviter la dégénérescence, des structures d'arbres **auto-équilibrés** ont été développées. Elles s'adaptent dynamiquement pour maintenir une hauteur ($h = O(\log n)$), garantissant des performances optimales.

Exemples :

- Arbres AVL
- Arbres rouges-noirs
- Arbres B

Ce qu’il faut retenir & Références

En bref : La notion de dégénérescence dans un ABR révèle ses limites sans mécanismes d’équilibrage. Cette contrainte majeure motive le développement des arbres équilibrés, essentiels pour des performances stables et efficaces sur de larges ensembles de données.

Sources consultées :

- GeeksforGeeks — BST degenerated
- Wikipedia — Binary Search Tree#Degenerate tree
- Programiz — BST Problems
- Coursera Algorithms — Tree height and balance